



AI Pentest

2025.12

传统渗透测试

极度依赖人工操作和专业知识，难以满足高效安全评估需求。

大语言模型渗透测试

大语言模型擅长使用测试工具执行复杂的命令和选项；

GPT-4 尤其擅长阅读源码并定位漏洞；

能够制作适当的测试命令，设计测试程序来寻找潜在漏洞。

存在**上下文丢失**问题，随着对话轮次增加，关键早期信息易丢失；

存在**命令幻觉**，生成不存在或参数错误的命令/工具，降低成功率；

知识局限性，无法覆盖最新漏洞信息或特点工具的复杂用法，存在知识滞后。

PentestGPT: An LLM-empowered Automatic Penetration Testing Tool

项目地址: <https://github.com/GreyDGL/PentestGPT>
测试视频: <https://www.youtube.com/watch?v=h0k6kWWaCEU>

LLM可以在多大程度上执行渗透测试任务？

- Finding 1: LLM熟练执行端到端渗透测试任务，但难以应对更困难的目标；
- Finding 2: LLMs可以有效地使用渗透测试工具，识别常见漏洞，并解释源代码以识别漏洞。

Table 1: Overall performance of LLMs on Penetration Testing Benchmark.

Tools	Easy		Medium		Hard		Average	
	Overall (7)	Sub-task (77)	Overall (4)	Sub-task (71)	Overall (2)	Sub-task (34)	Overall (13)	Sub-task (182)
GPT-3.5	1 (14.29%)	24 (31.17%)	0 (0.00%)	13 (18.31%)	0 (0.00%)	5 (14.71%)	1 (7.69%)	42 (23.07%)
GPT-4	4 (57.14%)	55 (71.43%)	1 (25.00%)	30 (42.25%)	0 (0.00%)	10 (29.41%)	5 (38.46%)	95 (52.20%)
Bard	2 (28.57%)	29 (37.66%)	0 (0.00%)	16 (22.54%)	0 (0.00%)	5 (14.71%)	2 (15.38%)	50 (27.47%)
Average	2.3 (33.33%)	36 (46.75%)	0.33 (8.33%)	19.7 (27.70%)	0 (0.00%)	6.7 (19.61%)	2.7 (20.5%)	62.3 (34.25%)

Table 2: Top 10 Types of Sub-tasks completed by each tool.

Sub-Tasks	WT	GPT-3.5	GPT-4	Bard
Web Enumeration	18	4 (22.2%)	8 (44.4%)	4 (22.2%)
Code Analysis	18	4 (22.2%)	5 (27.2%)	4 (22.2%)
Port Scanning	12	9 (75.0%)	9 (75.0%)	9 (75.0%)
Shell Construction	11	3 (27.3%)	8 (72.7%)	4 (36.4%)
File Enumeration	11	1 (9.1%)	7 (63.6%)	1 (9.1%)
Configuration Enumeration	8	2 (25.0%)	4 (50.0%)	3 (37.5%)
Cryptanalysis	8	2 (25.0%)	3 (37.5%)	1 (12.5%)
Network Enumeration	7	1 (14.3%)	3 (42.9%)	2 (28.6%)
Command Injection	6	1 (16.7%)	4 (66.7%)	2 (33.3%)
Known Exploits	6	2 (33.3%)	3 (50.0%)	1 (16.7%)

人类渗透测试员和LLM的问题解决策略有何不同？

- Finding 3: LLMs很难维持长期记忆，而这对有效链接漏洞和制定利用策略至关重要；
- Finding 4: LLMs倾向最近的任务和深度优先搜索，会导致过度关注某一任务并忘记之前的发现；
- Finding 5: LLMs可能会产生不准确的操作或命令。

Table 3: Top Unnecessary Operations Prompted by LLMs on the Benchmark Targets

Unnecessary Operations	GPT-3.5	GPT-4	Bard	Total
Brute-Force	75	92	68	235
Exploit Known Vulnerabilities (CVEs)	29	24	28	81
SQL Injection	14	21	16	51
Command Injection	18	7	12	37

Table 4: Top causes for failed penetration testing trials

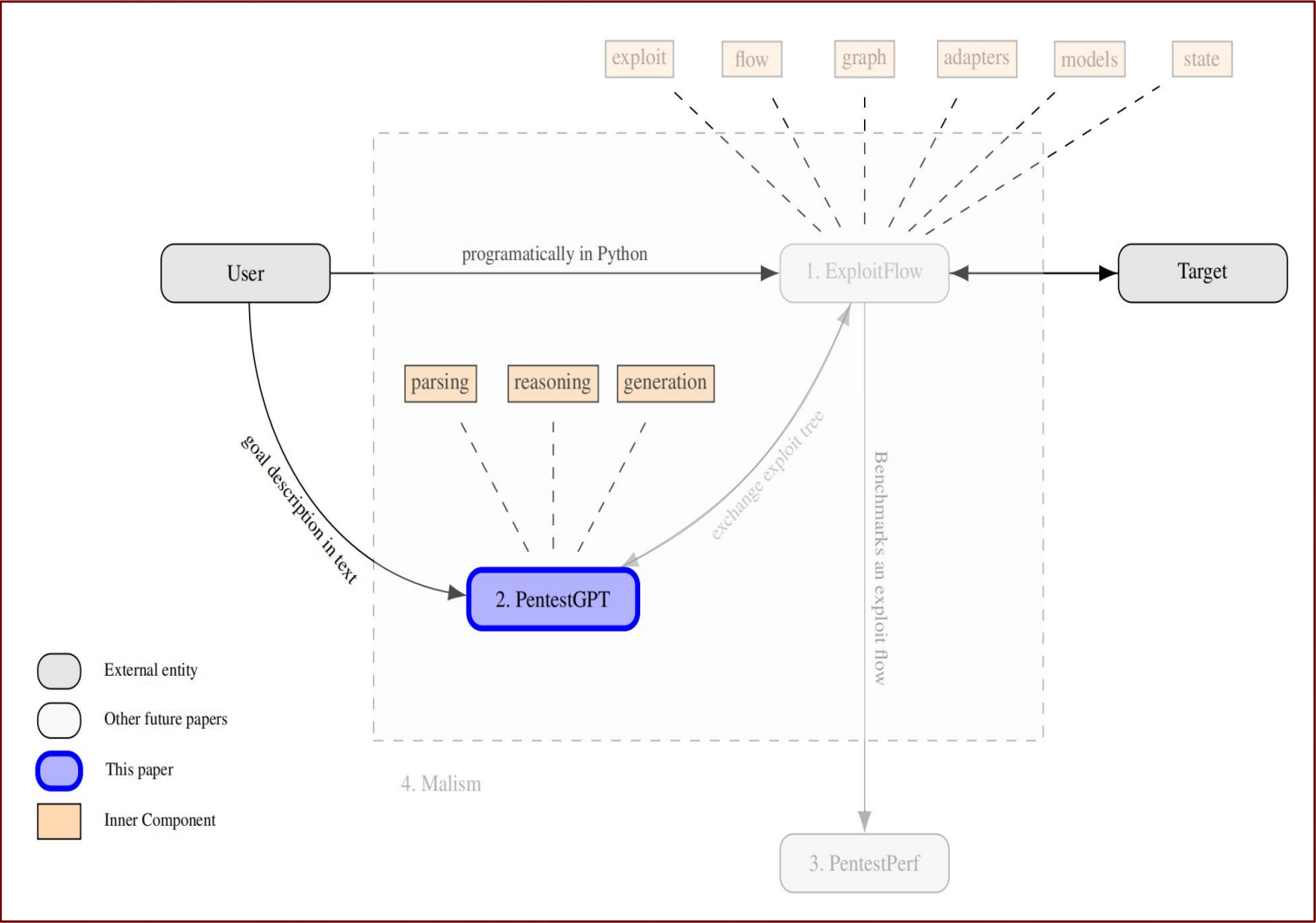
Failure Reasons	GPT3.5	GPT4	Bard	Total
Session context lost	25	18	31	74
False Command Generation	23	12	20	55
Deadlock operations	19	10	16	45
False Scanning Output Interpretation	13	9	18	40
False Source Code Interpretation	16	11	10	37
Cannot craft valid exploit	11	15	8	34

13 台真实靶机 × 182 原子子任务 × 18 种 CWE 的渗透测试基准

作者从HackTheBox 和 VulnHub 中选择靶机，涵盖OWASP Top 10项目中列出的所有漏洞，并按照渗透测试领域的传统标准分为简单、中等和困难类别，构建渗透测试基准

Phase	Technique	Description	Related CWEs
Reconnaissance	Port Scanning	Identify the open ports and related information on the target machine.	CWE-668
	Web Enumeration	Gather detailed information about the target's web applications.	
	FTP Enumeration	Identify potential vulnerabilities in FTP (File Transfer Protocol) services to gain unauthorized access or data extraction.	
	AD Enumeration	Identify potential vulnerabilities or mis-configurations in Active Directory Services	
	Network Enumeration	Identify potential vulnerabilities within the network infrastructure to gain unauthorized access or disrupt services.	
	Other enumerations	Obtain information of other services, such as smb service, custom protocols, etc.	
Exploitation	Command Injection	Inject arbitrary commands to be run on a host machine, often leading to unauthorized system control.	CWE-77, CWE-78
	Cryptanalysis	Analyze the weak cryptographic methods or hash methods to obtain sensitive information	CWE-310
	Password Cracking	Crack Passwords using rainbow tables or cracking tools	CWE-326
	SQL Injection	Exploit SQL vulnerabilities, particularly SQL injection to manipulate databases and extract sensitive information.	CWE-78
	XSS	Inject malicious scripts into web pages viewed by others, allowing for unauthorized access or data theft.	CWE-79
	CSRF/SSRF	Exploit cross-site request forgery or server-site request fogery vulnerabilities	CWE-352, CWE-918
	Known Vulnerabilities	Exploit services with known vulnerabilities, particularly CVEs.	CWE-1395
	XXE	Exploit XML extenal entitiy vulnerabilities to achieve code execution.	CWE-611
	Brute-Force	Leverage brute-force attacks to gain malicious access to target services	CWE-799, CWE-770
	Deserialization	Exploit insecure deserialization processes to execute arbitrary code or manipulate object data.	CWE-502
Privilege Escalation	Other Exploitations	Other exploitations such as AD specific exploitation, prototype pollution, etc.	
	File Analysis	Enumerate system/service files to gain malicious information for privilege escalation	CWE-200, CWE-538
	System Configuration Analysis	Enumerate system/service configurations to gain malicious information for privilege escalation	CWE-15, CWE-16
	Cronjob Analysis	Analyze and manipulate scheduled tasks (cron jobs) to execute unauthorized commands or disrupt normal operations.	CWE-250
	User Access Exploitation	Exploit the improper settings of user access in combination with system properties to conduct privilege escalation	CWE-284
General Techniques	Other techniques	Other general techniques, such as exploiting running processes with known vulnerabilities	
	Code Analysis	Analyze source codes for potential vulnerabilities	
	Shell Construction	Craft and utilize shell codes to manipulate the target system, often enabling control or extraction of data.	
	Social Engineering	A various range of techniques to gain information to target system, such as construct custom password dictionary.	
	Others	Other techniques	

作者提出了一个全面的自动化框架 MALISM

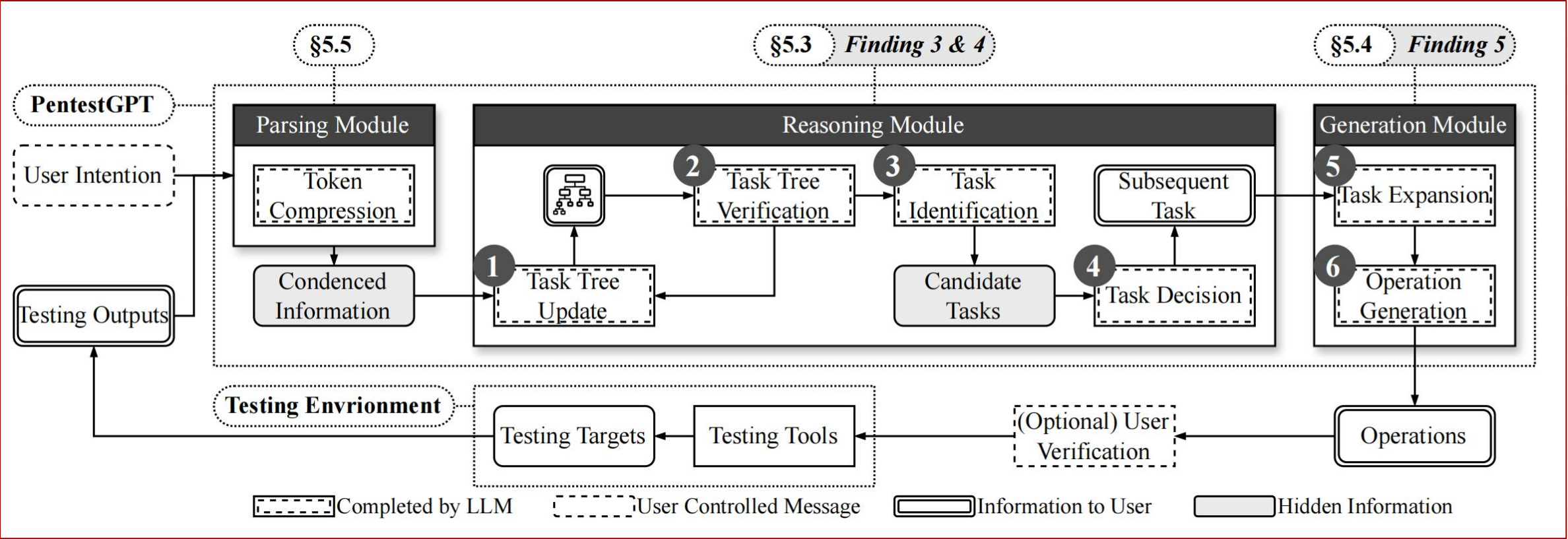


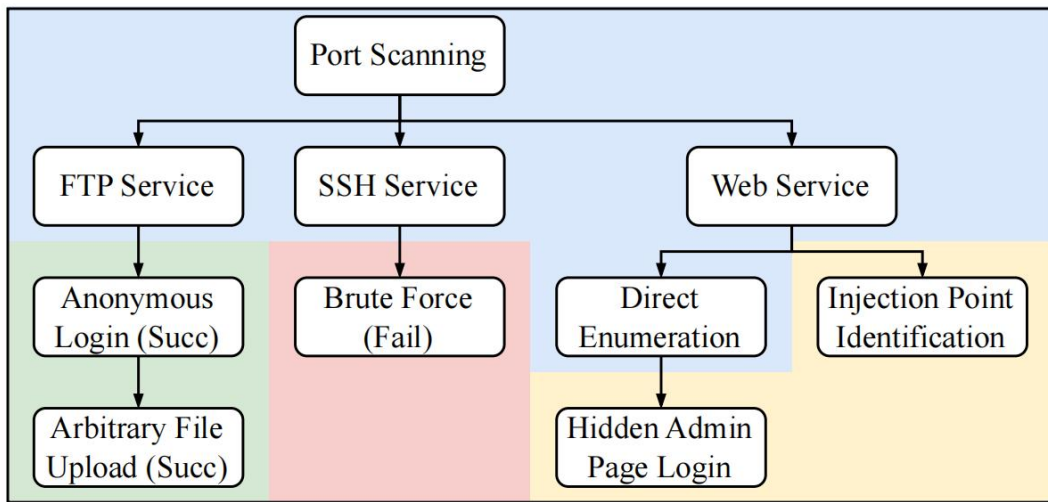
EXPLOITFLOW:
通过捕获每个离散动作后的系统状态来生成安全开发路线

PentestGPT:
利用LLMs为每个给定的离散状态启发式生成测试指导

PentestPerf:
综合渗透测试基准，用于评估渗透测试器和自动化工具在广泛的测试目标上的性能

先验研究表明LLMs存在全局上下文易丢失、近期偏差以及命令幻觉的问题。针对该发现，作者设计了交互式系统PENTESTGPT，一方面优化LLMs在渗透测试中的使用，另一方面利用LLMs的优势提高自动渗透测试的效率和有效性。





a) PTT Representation

Task Tree:

1. Perform port scanning (completed)
 - Port 21, 22 and 80 are open.
 - Services are FTP, SSH, and Web Service.
2. Perform the testing
 - 2.1 Test FTP Service
 - 2.1.1 Test Anonymous Login (success)
 - 2.1.1.1 Test Anonymous Upload (success)
 - 2.2 Test SSH Service
 - 2.2.1 Brute-force (failed)
 - 2.3 Test Web Service (ongoing)
 - 2.3.1 Directory Enumeration
 - 2.3.1.1 Find hidden admin (to-do)
 - 2.3.2 Injection Identification (todo)

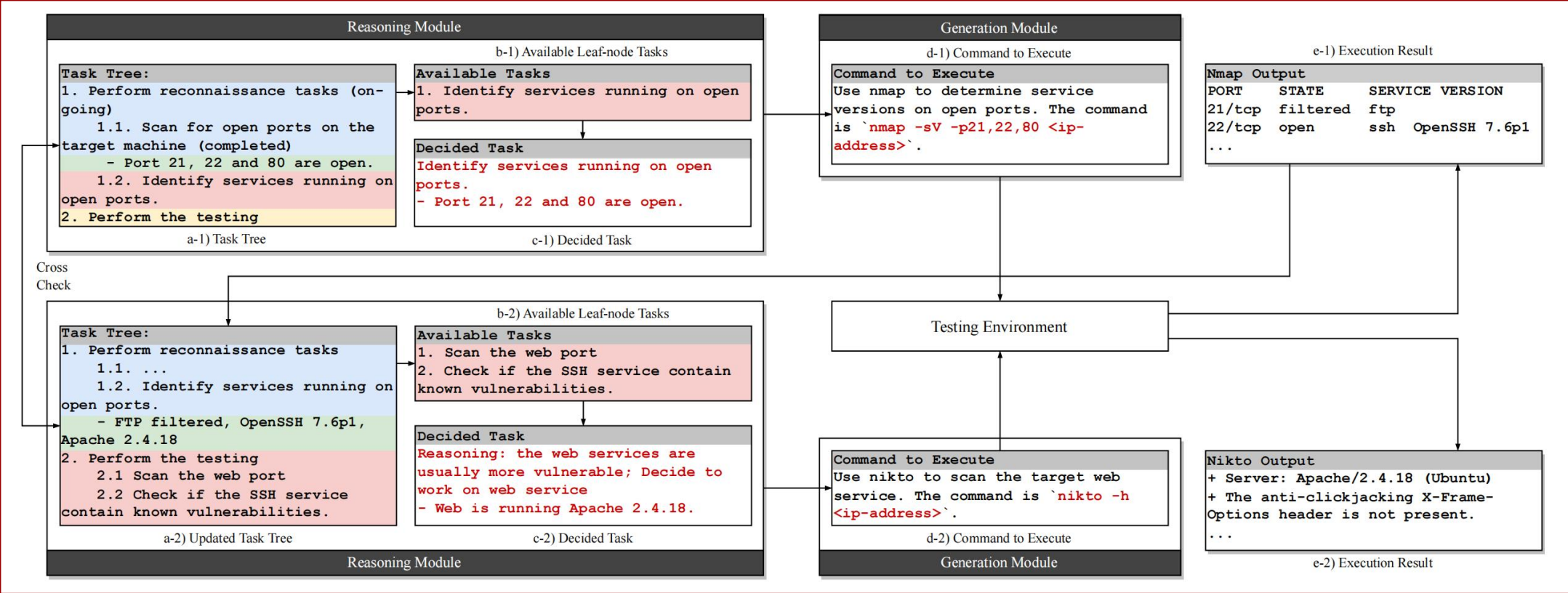
b) PTT Representation in Natural Language

负责根据解析后的信息生成渗透测试执行策略

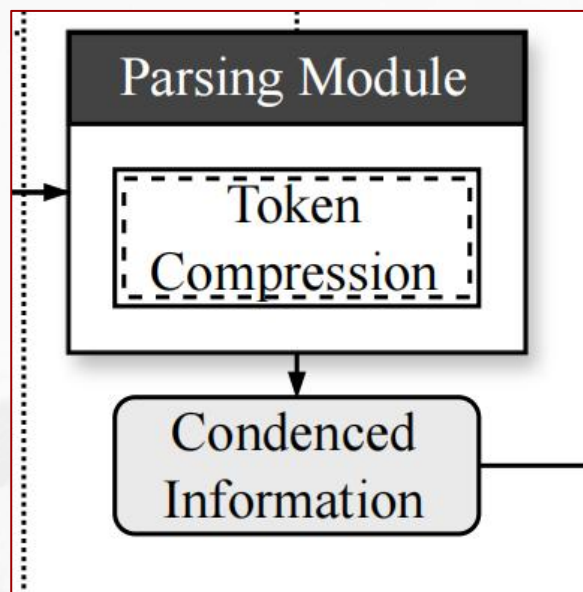
- **初始PTT构建：**作者针对渗透测试引入了一种新的表示Pentesting Task Tree（见左图），利用树形结构，层次化表示测试任务，有效克服了记忆丢失问题，还可以将任务转化为自然语言的描述，从而使渗透测试报告更加易懂。
- **PTT验证更新：**对新更新的PTT进行验证，确保只有叶节点被修改，防止LLM因幻觉而对整体树结构进行潜在更改。
- **候选任务识别：**确定可行的候选子任务。
- **任务优先级排序与推荐：**评估这些子任务导致成功渗透测试结果的可能性，并推荐最高优先级的任务作为输出。

负责将推理模块传递的具体子任务转化为具体的命令或指令，采用了CoT（思维链）策略。

- 子任务扩展：将收到推理模块传递的子任务扩展为一系列详细步骤
- 步骤转化为命令：将步骤转化为精确的终端命令执行，或者转化为特定图形用户界面操作描述。



负责把渗透测试过程中产生的大量原始输出整理成 Reasoning Module能直接利用的关键信息

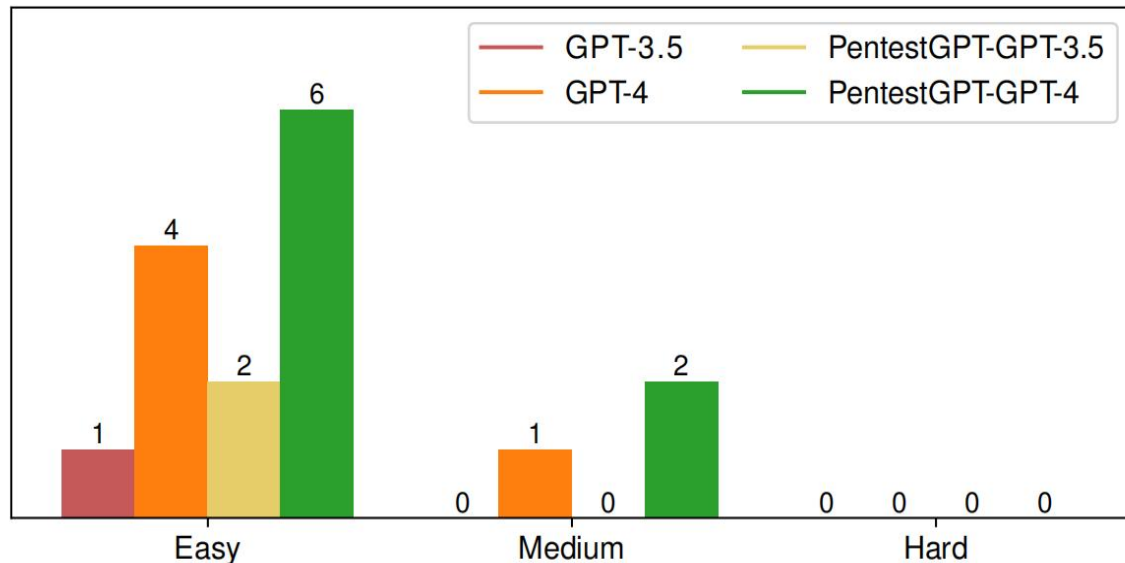


- 用户意图——把用户输入的自然语言指令压缩成系统能理解、能交给Reasoning的具体信息；
- 安全工具输出——对nmap、nikto、gobuster、hydra等工具的冗长输出做摘要，提取真正有价值的部分；
- HTTP页面内容——对网页代码、返回包、错误页等进行整理，提取可能包含敏感线索的字段
- 源代码片段——对渗透测试中获得的Python、PHP、JS等代码进行分析，并配合GPT-4 code interpreter做静态检查。

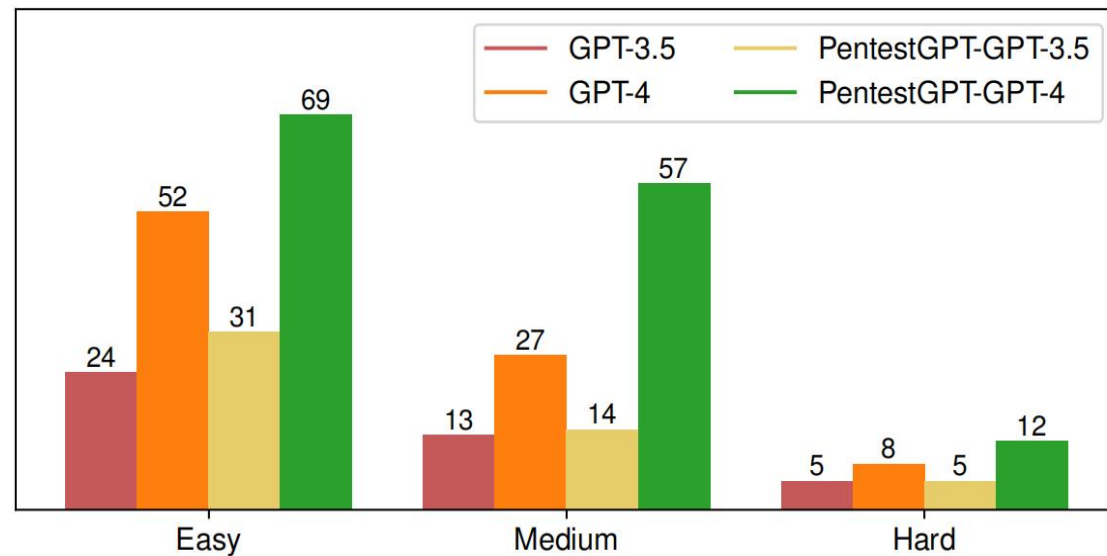
作者使用1,900行Python3代码和740行提示词实现了PENTESTGPT，对其在构建的基准以及额外的真实世界渗透测试机器上的性能进行了评估。将PENTESTGPT与GPT-3.5和GPT-4集成，形成了两个工作版本。

如图所示，在整体任务完成情况以及子任务完成情况中，与GPT-3.5和GPT-4的直接使用相比，应用PENTESTGPT架构比LLMs原生应用均具备更优越的渗透测试能力。

与原生GPT-4相比，PENTESTGPT-GPT-4还完成了更多子任务（57 vs 27），增幅达111%。



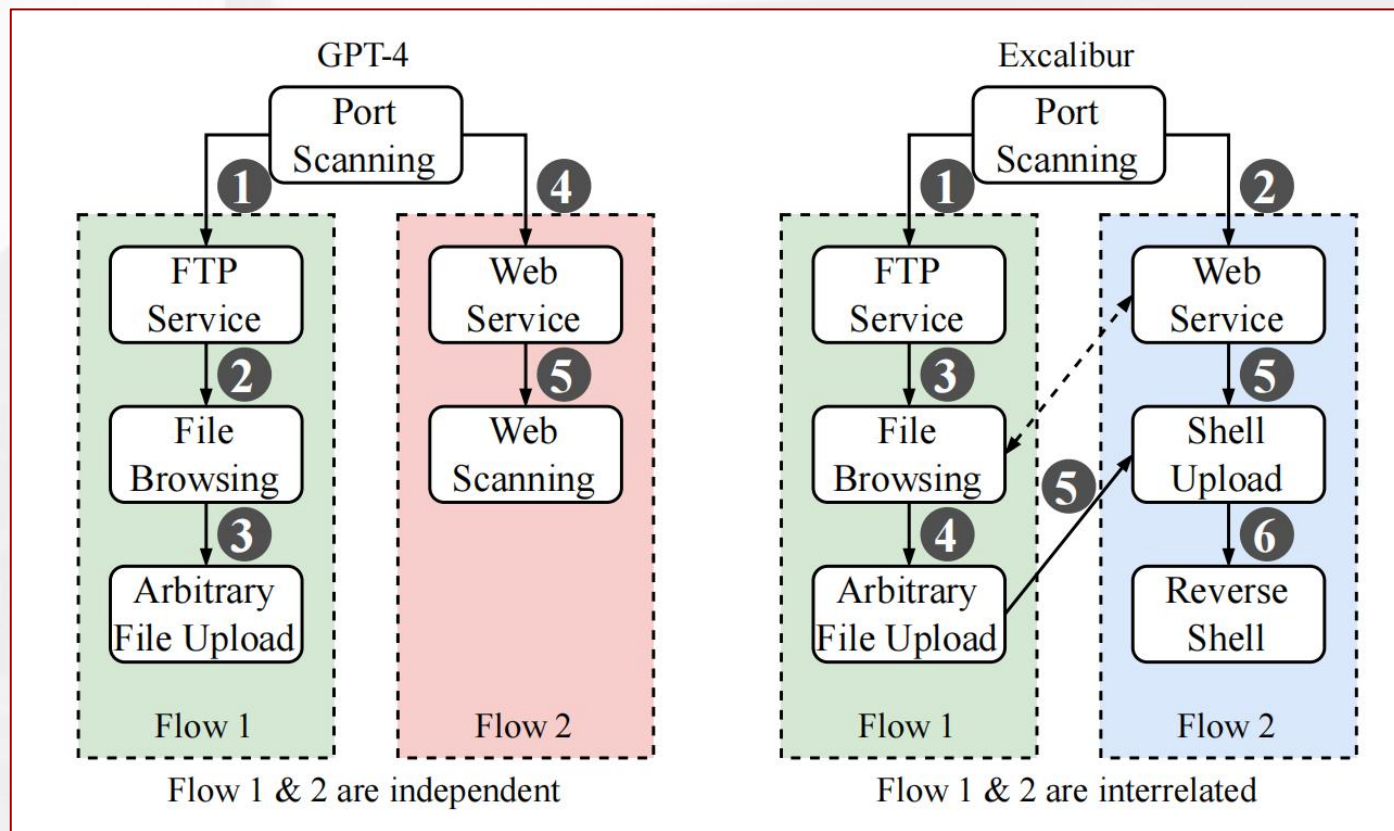
(a) Overall completion status.



(b) Subtask completion status.

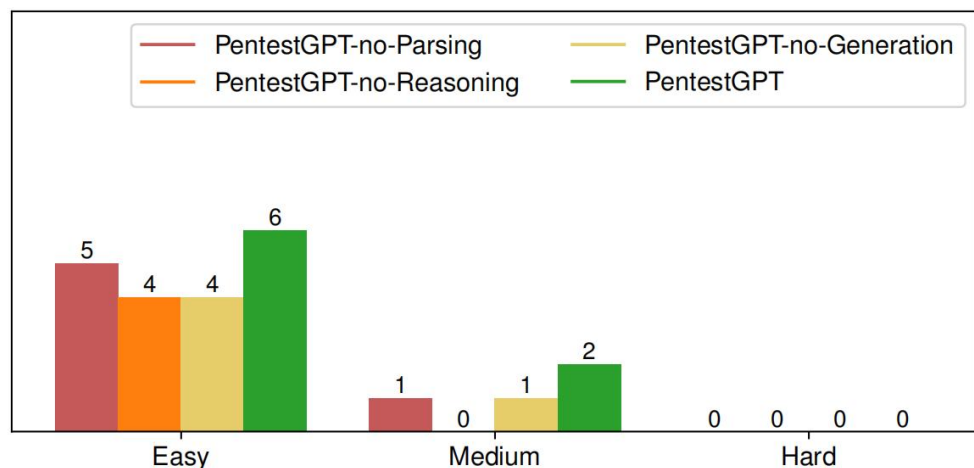
作者将PENTESTGPT与LLMs和人类专家进行了比较，对比了GPT-4和PENTESTGPT在包含两个漏洞的机器上进行渗透测试时的策略。

相比之下，PENTESTGPT可以在FTP和Web服务之间切换，较GPT-4而言找到了两服务之间的联系，以类似于人类专家的方式分解渗透测试任务，从而成功实现了总体目标。

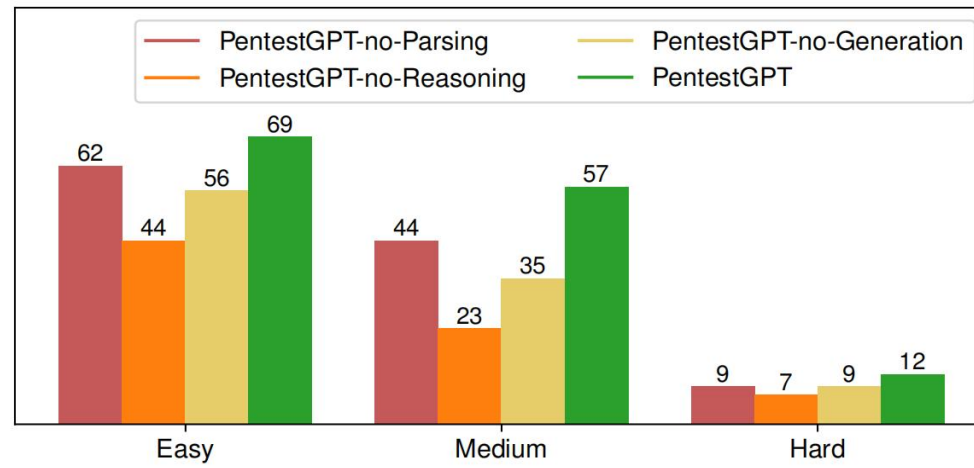


作者通过依次移除三个模块（Parsing、Generation、Reasoning）来观察系统性能变化

- NO-PARSING：移除解析模块后，系统直接接收工具的原始输出，虽然依然能够运行，但噪声明显增加，任务推进效率下降
- NO-GENERATION：移除生成模块后，由Reasoning模块直接输出命令，结果导致命令质量不稳定，错误率升高，影响整个渗透流程的连贯性。
- NO-REASONING：去掉Reasoning模块及其核心的任务树机制后，系统退化为普通LLM，表现与GPT-4几乎无异，完整渗透率大幅下降。



(a) Overall completion status



(b) Sub-task completion status

作者将PENTESTGPT应用于HackTheBox以及picoMini 挑战，将PENTESTGPT与GPT-4 32k token长度的API结合使用。

HackTheBox中以131.5美元成本完成10个选定目标中的5个，能以合理的成本有效完成任务。

picoMini 中成功解决了21个挑战中的9个，在248支提交有效结果的团队中排名第24位

表明PENTESTGPT在各种类型的现实世界渗透测试任务中表现出色

Machine	Difficulty	Completions	Completed Users	Cost (USD)
Sau	Easy	5/5 (✓)	4798	15.2
Pilgrimage	Easy	3/5 (✓)	5474	12.6
Topology	Easy	0/5 (✗)	4500	8.3
PC	Easy	4/5 (✓)	6061	16.1
MonitorsTwo	Easy	3/5 (✓)	8684	9.2
Authority	Medium	0/5 (✗)	1209	11.5
Sandworm	Medium	0/5 (✗)	2106	10.2
Jupiter	Medium	0/5 (✗)	1494	6.6
Agile	Medium	2/5 (✓)	4395	22.5
OnlyForYou	Medium	0/5 (✗)	2296	19.3
Total	-	17/50 (6)	-	131.5

Challenge	Category	Score	Completions
login	web	100	5/5 (✓)
advance-potion-making	forensics	100	3/5 (✓)
spelling-quiz	crypto	100	4/5 (✓)
caas	web	150	2/5 (✓)
XtrOrdinary	crypto	150	5/5 (✓)
tripplesecure	crypto	150	3/5 (✓)
clutteroverflow	binary	150	1/5 (✓)
not crypto	reverse	150	0/5 (✗)
scrambled-bytes	forensics	200	0/5 (✗)
breadth	reverse	200	0/5 (✗)
notepad	web	250	1/5 (✓)
college-rowing-team	crypto	250	2/5 (✓)
fermat-strings	binary	250	0/5 (✗)
corrupt-key-1	crypto	350	0/5 (✗)
SaaS	binary	350	0/5 (✗)
risky business	reverse	350	0/5 (✗)
homework	binary	400	0/5 (✗)
lockdown-horses	binary	450	0/5 (✗)
corrupt-key-2	crypto	500	0/5 (✗)
vr-school	binary	500	0/5 (✗)
MATRIX	reverse	500	0/5 (✗)



北京大學
PEKING UNIVERSITY

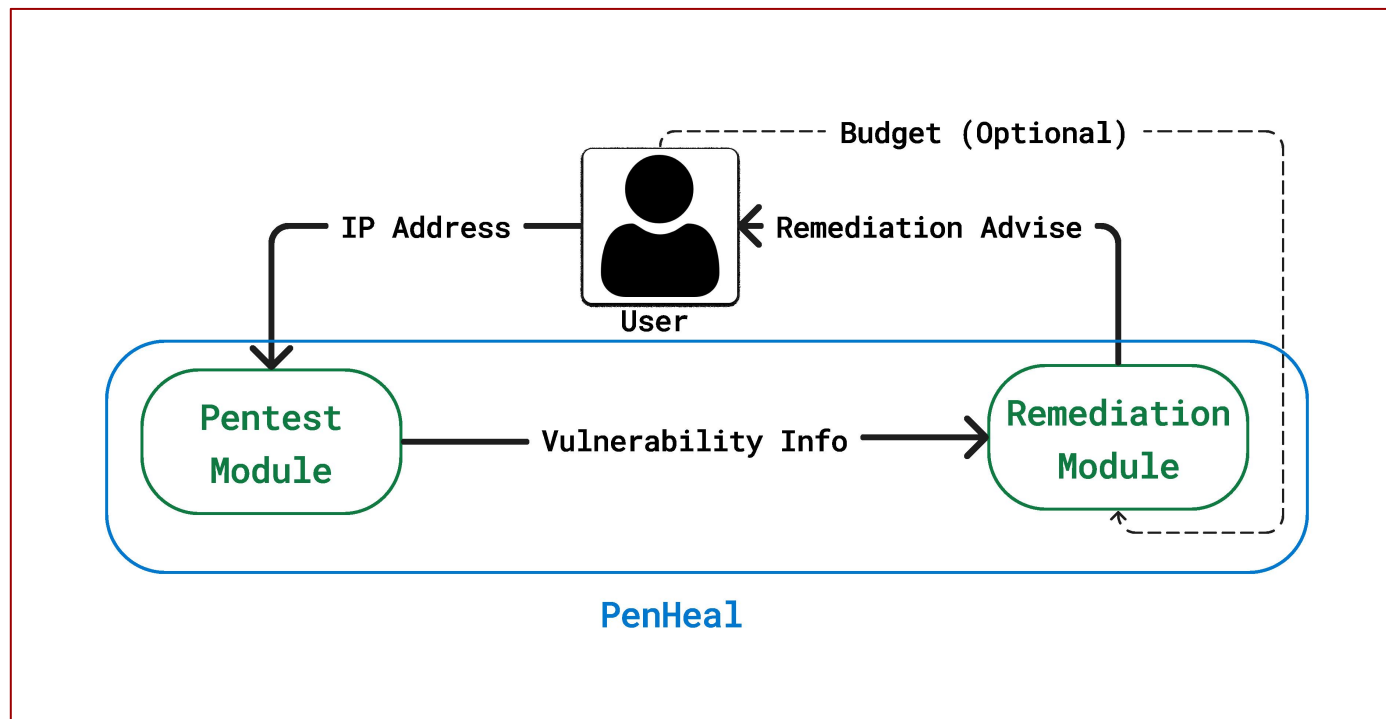
PenHeal: A Two-Stage LLM Framework for Automated Pentesting and Optimal Remediation

项目复现地址: https://github.com/Thaopieh/NT140.P11.ANTT-Final_Project

目前大多智能渗透测试工作止步于漏洞扫描，后续修复环节仍由人工评估、决策和执行，缺乏自动化闭环。作者提出了 **PenHeal**，一种新颖的基于LLM的自动化渗透测试与漏洞修复框架。框架包括两阶段 LLM 流程，并通过模块化设计实现了高覆盖率、低成本、高效能的漏洞检测与修复。

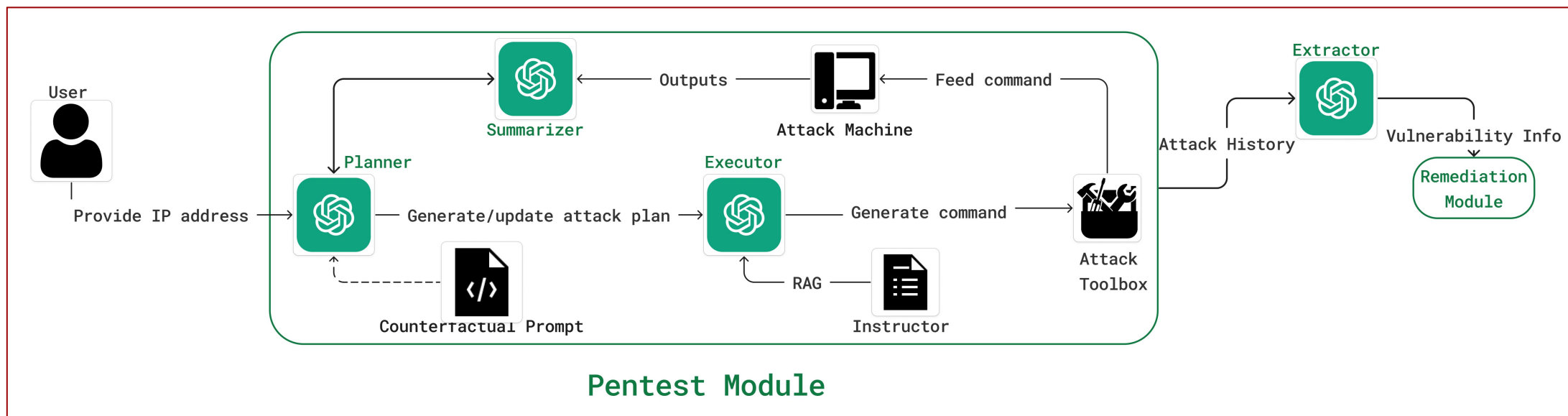
传统智能渗透测试方法只能实现单点爆破，针对某个漏洞进行发现、利用、提权的操作，导致大量剩余漏洞被遗漏并且无对应的具体修复方案。PenHeal重点提示漏洞覆盖率，首次用 LLM 把“渗透”与“修复”连成一条无人值守、预算可控、效果可量化的自动化管道，给出了预算最优的修复方案。

PenHeal包含两阶段模块：**渗透测试模块**和**修复建议模块**。
用户首先将目标受害者机器的信息输入到渗透测试模块后，渗透测试模块扫描系统并将发现的漏洞传递给修复模块。
用户可选择以纯文本形式设置不同类型修复方法的成本和总体预算。若没有选择将应用默认设置，向用户展示一组最优的修复策略。



用户输入目标IP地址，Planner 制定攻击策略，将任务结构化排列，Executor 在 Instructor指导下生成命令，同时利用反事实提示驱动 Planner 迭代更新任务树，优化攻击策略，最后由 Extractor 输出标准化漏洞集合

- **Planner**: 用实时更新的任务树(to-do / done / failed) 来规划下一步；当某漏洞被成功利用，立即触发反事实提示，假装该漏洞不存在，从而强制探索新路径。
- **Executor & Instructor**: Executor 将 Planner 的任务生成可执行命令；Instructor 通过 RAG 实时检索外部知识库，提供最相关的用法片段，减少幻觉、提升命令正确率。
- **Summarizer & Extractor**: Summarizer 总结压缩Executor输出，Extractor 再把它格式化输出为 “Exploited: [backdoor/CVE ID]” 的统一漏洞记录，作为下一阶段输入。

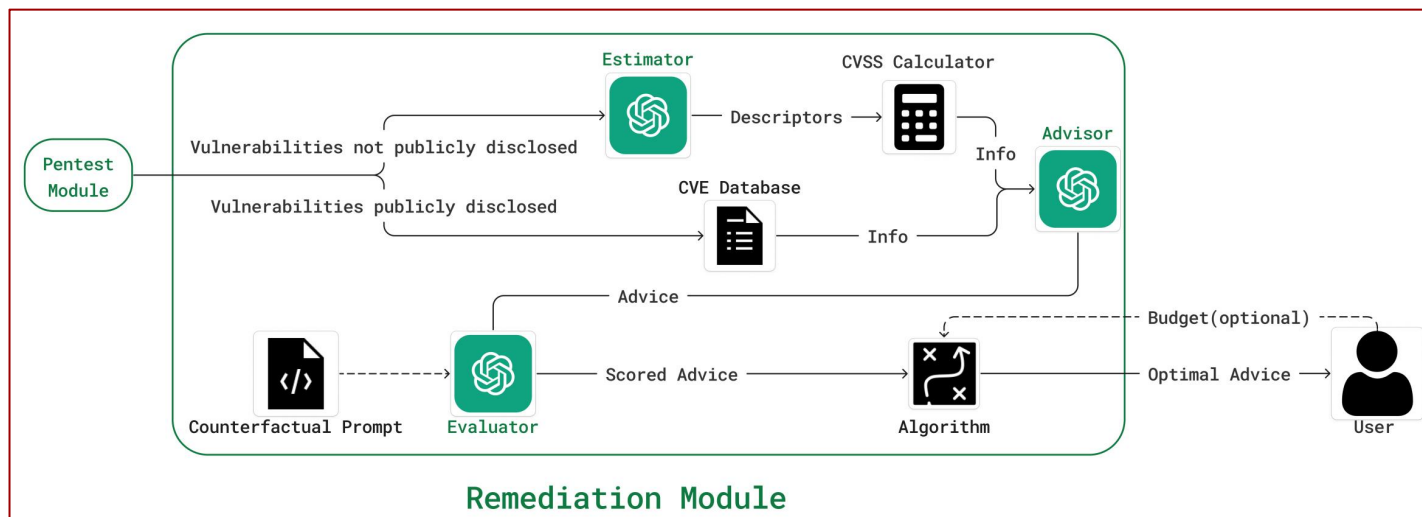


为漏洞检索额外属性，或根据可用数据描述评估其严重性和属性。在用户定义的预算约束下，输出最优修复方案。

- **Estimator**: 为每个漏洞生成 CVSS 向量——已知漏洞直接查询国家漏洞数据库，未知漏洞由 LLM 调用开源计算器，得到统一的风险分数。
- **Advisor**: 依据漏洞类型、服务版本生成多条修复建议（系统更新、补丁、配置更新等）。
- **Evaluator**: 在预算约束下做最优修复策略选择。利用分组背包算法在用户给定预算内求出价值最大化的修复子集，输出最终可执行清单。

Value: 若建议能完全解决某漏洞，则 value 为该漏洞的 CVSS 分；若只能部分解决，则为 CVSS 的 k%；若不能解决或会恶化，则 value 为 0 或负数。

Cost: 人工或默认值根据时间、金钱等因素把每条建议映射成 0–10 的分数。



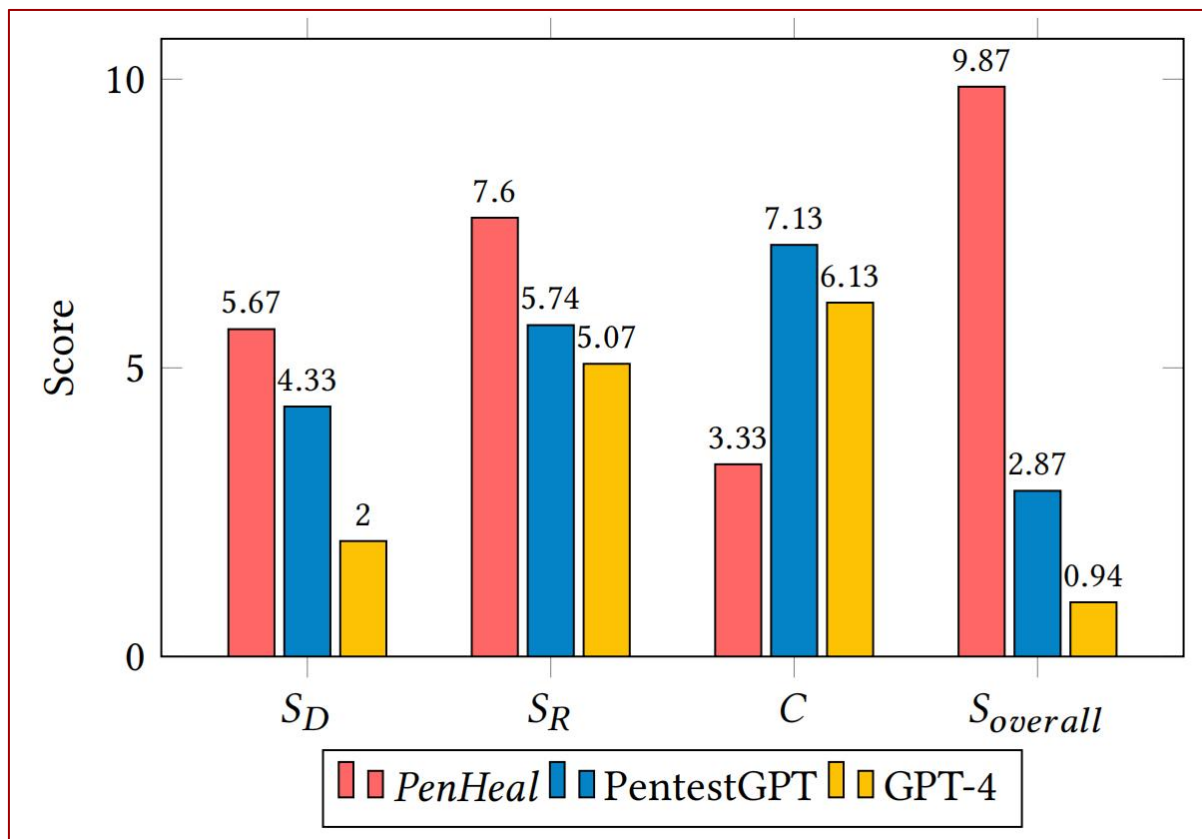
测试环境

- 靶机：Metasploitable2 Linux 虚拟机（存在 10 类可利用漏洞）
- 攻击机：Kali Linux 虚拟机
- LLM：Planner/Executor/Advisor/Evaluator 用 GPT-4； Summarizer/Extractor/Estimator 用 GPT-3.5-turbo； Instructor 用 LangChain-RAG。

评分指标

- 检测覆盖率得分 S_D ：检测到的漏洞数量占该环境中总漏洞数量的百分
- 修复有效性评分 S_R ：模型生成的所有修复建议value评分的平均值
- 修复成本评分 C ：模型生成的所有修复建议cost评分的平均值
- 综合能力 $S_{overall}$ ：
$$S_{overall} = \frac{S_D + S_R - C}{3}$$

文章将模型 PenHeal 与两个稳定的基线模型 PentestGPT 和 GPT-4 进行基准测试来评估其有效性。PentestGPT 与 GPT-4 都需要人工介入解析输出或切换任务；而 PenHeal 只需提供靶机 IP，整个过程完全自动化



- 检测覆盖率: PenHeal 5.67, 比 GPT-4 高 31%
- 修复有效性: PenHeal 7.6, 比 Pentest GPT 高 32%
- 修复成本: PenHeal 2.87, 比 GPT-4 低 46%。
- 综合得分: PenHeal 9.87, 显著领先。



北京大學
PEKING UNIVERSITY

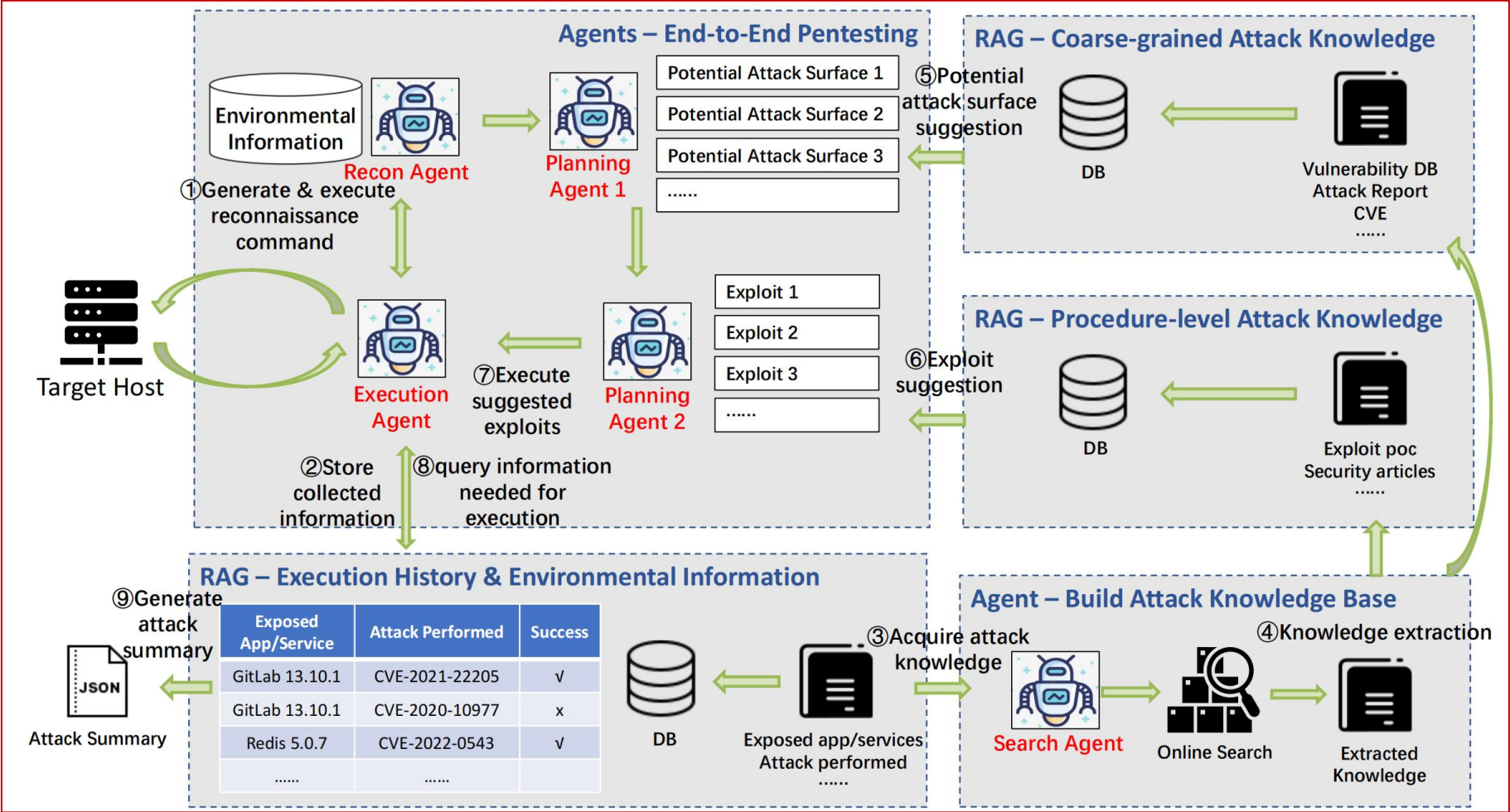
PentestAgent: Incorporating LLM Agents to Automated Penetration Testing

项目地址: <https://github.com/nbshenxm/pentest-agent>

基于上述传统渗透测试和大语言模型渗透测试的局限性，作者提出了PentestAgent，一种新颖的基于LLM的自动化渗透测试框架，该框架利用了LLM的能力以及RAG检索增强生成技术，实现端到端的自动化渗透测试。

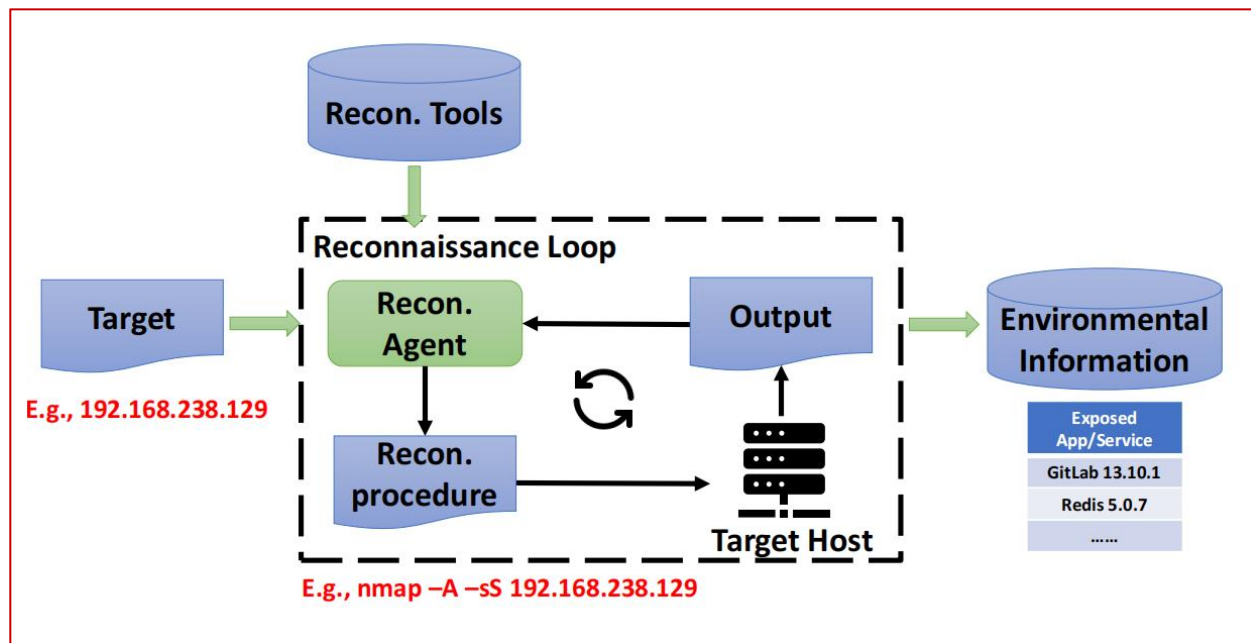
PentestAgent可以被视为对PentestGPT的系统性改进，PentestGPT是LLM辅助测试，需要人工逐轮反馈，而本文中的PentestAgent则减少了人工干预，智能体自主完成信息收集、漏洞分析、利用阶段。此外，本文也通过实验证明了PentestAgent在任务完成率和效率上均优于PentestGPT。

PentestAgent 包含四个主要组件：侦察代理、搜索代理、规划代理、执行代理，这些代理相互协作完成渗透测试的三个主要阶段：情报收集、漏洞分析和漏洞利用



接收指定目标作为输入，并与其交互以收集详细信息，最终生成环境信息摘要作为输出

- 使用CoT将复杂任务分解为多个子任务，并构建有效的侦察工作流程减少幻觉现象。
- 采用RAG机制，可以从包含各种侦察工具文档的数据库中检索相关信息，并将收集到的环境信息存储在数据库中供以后使用



Reconnaissance System Message (Simplified)

Role-play

You're an excellent cybersecurity penetration tester assistant. Guide the tester ...

Chain-of-Thought

Use Nmap to identify exposed ports, then use relevant tools in Nmap to analyze these ports on the target host ...

RAG

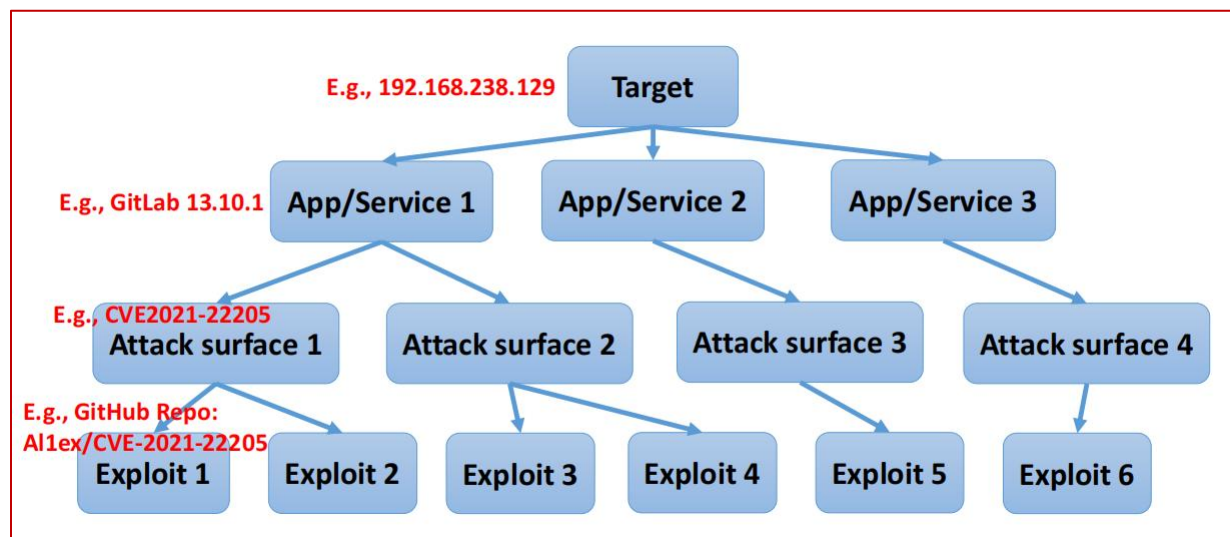
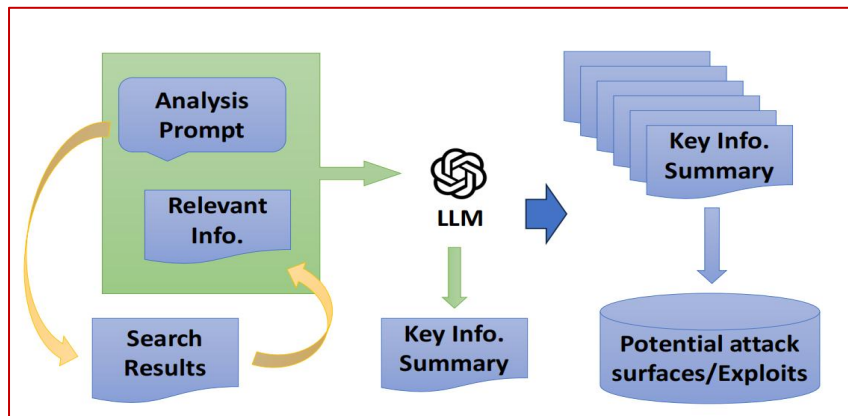
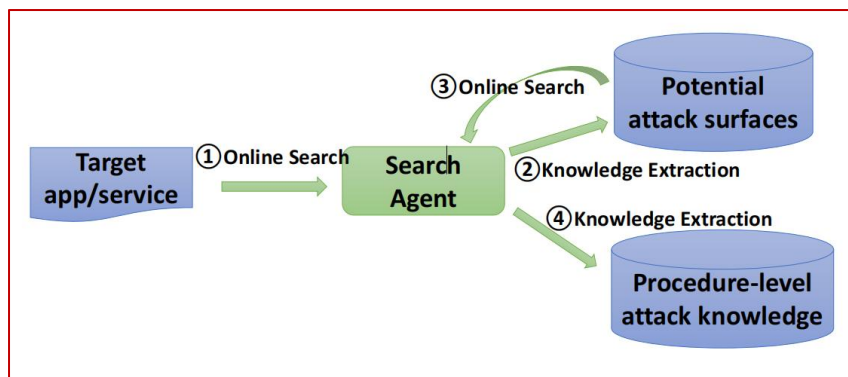
You should use your query tool to learn about available reconnaissance tools ...

Structured Output

You should always respond in valid JSON format with the following fields: {FORMAT SPEC.} ...

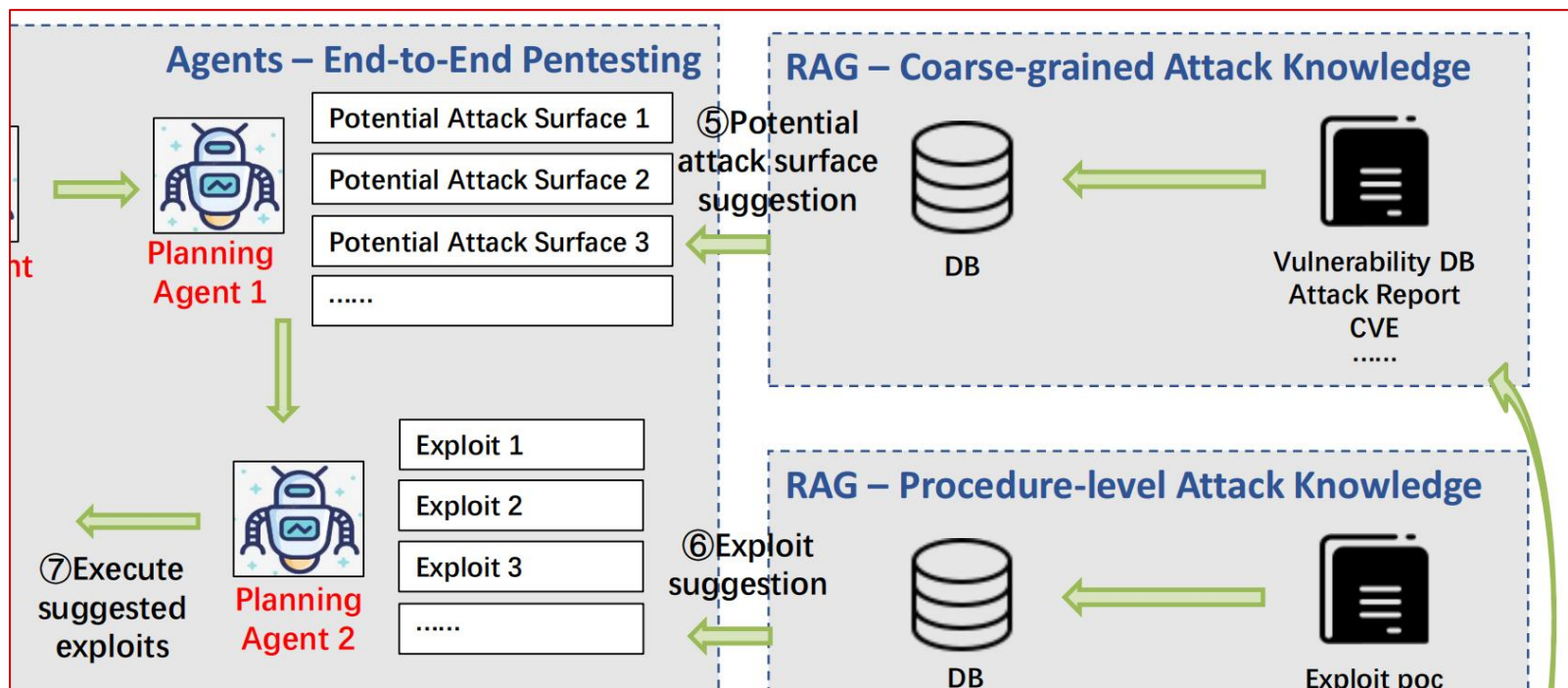
将目标服务和应用程序作为输入，并将相关攻击知识存储到数据库中作为输出。

- 第一轮搜索：搜索潜在攻击面，从 Google、Snyk 等数据库提取信息
- 后续轮搜索：基于攻击面搜索过程级攻击知识（如 ExploitDB、GitHub 上的漏洞利用代码）
- 知识提取：RAG机制，向量检索，进行关键信息提取，再以结构化格式存储到分层知识库



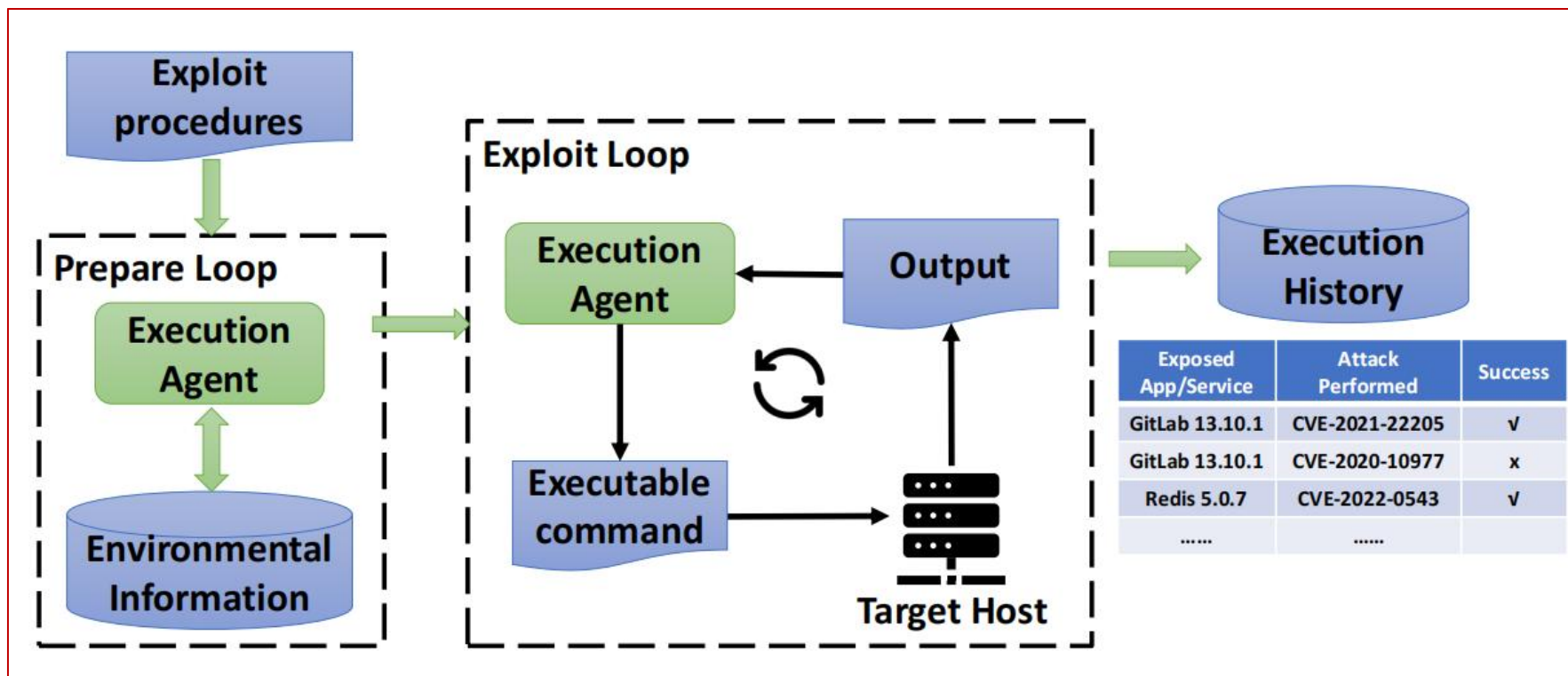
以侦察代理检测到的服务和应用程序作为输入，并生成漏洞利用计划作为输出。

- 调用 RAG 查询知识库，根据目标服务或应用的版本和漏洞类型，筛选潜在攻击面
- 基于攻击面进一步匹配漏洞利用工具，并按利用效果分类排序，生成可执行的利用计划。

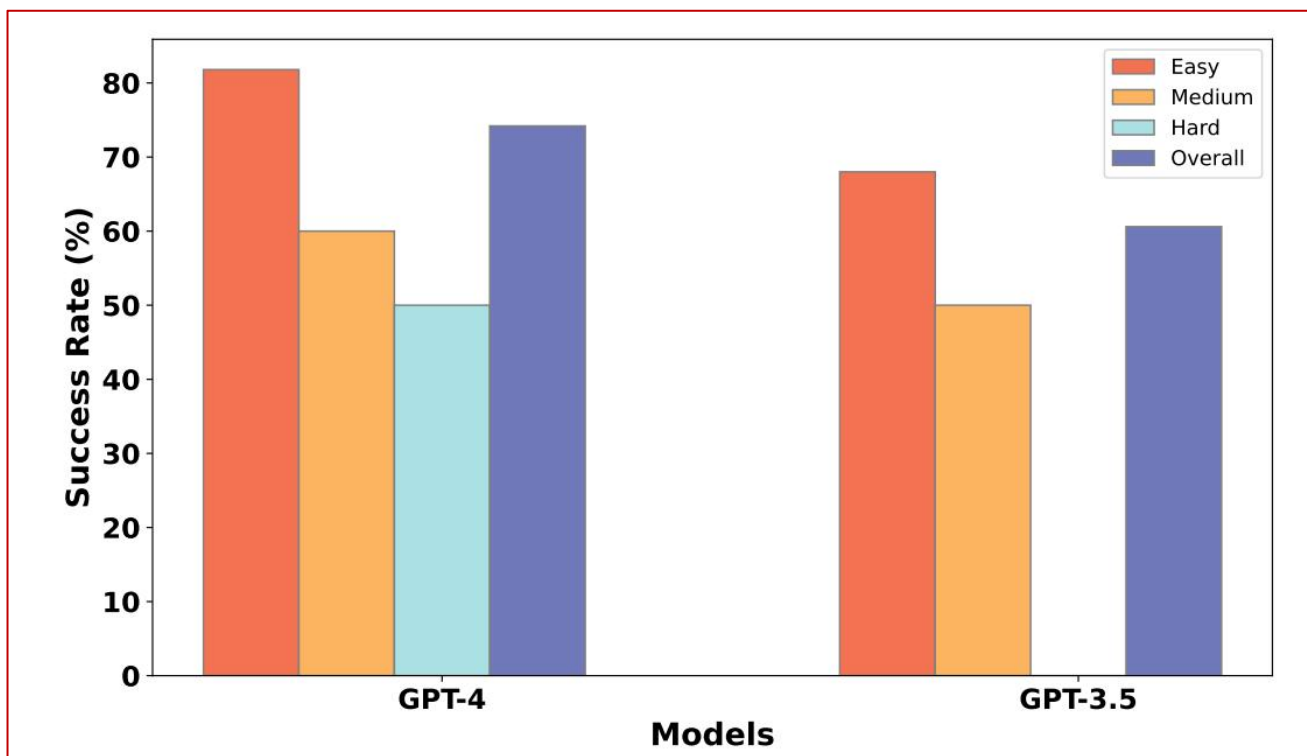


以漏洞利用细节作为输入，尝试在目标上自动执行漏洞利用，并最终生成漏洞利用摘要作为输出

- 准备阶段：分析漏洞利用所需参数，查询环境信息数据库补充缺失信息
- 利用阶段：生成分步执行指南，调用工具执行；采用自我反思技术，分析修复错误并记录错误历史，避免重复错误



选择 VulHub 作为基准数据集，制作了一个包含 67 个渗透测试目标的基准测试，涵盖 32 个 CWE（常见弱点枚举）类别，有 50 个目标属于易利用难度，11 个目标具有中等可利用难度，6 个目标具有高可利用难度。

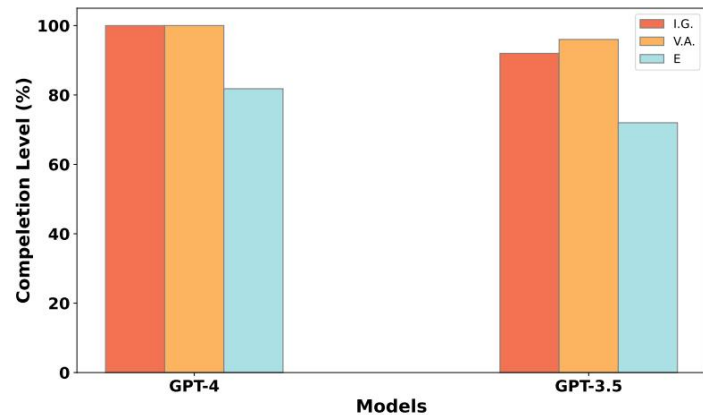


- GPT-4 模型在完成自动化渗透测试任务方面表现出 74.2% 的总体成功率，优于 GPT-3.5 模型的 60.6%
- 两种模型的总成功率均超过 60%，PentestAgent 有效性得以证明
- 性能差异不显著。PentestAgent 并未过度依赖 LLM 的通用知识和能力。

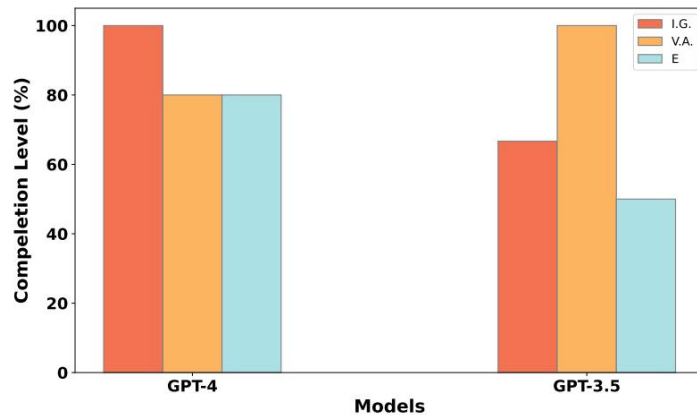
GPT-4: 简单任务中, 情报收集和漏洞分析阶段实现完全完成, 漏洞利用阶段完成率达 81.8%; 中等难度任务中, 漏洞分析阶段成功率略有下降; 高难度任务中情报收集阶段完成率降至 50%, 在处理需要更好侦察工具和高级推理能力的复杂场景时仍存在局限性。

GPT-3.5: 简单任务中, 情报收集 (92%) 和漏洞分析 (96%) 阶段完成率较高, 但漏洞利用阶段 (72%) 稍显不足; 中等难度任务中, 漏洞分析阶段虽保持 100% 完成率, 但情报收集 (66.7%) 和漏洞利用 (50%) 表现下滑; 高难度任务中, 情报收集和漏洞利用阶段均未完成。

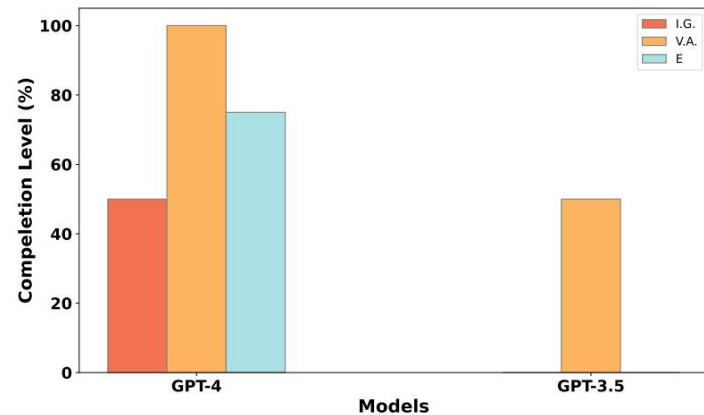
GPT-4 模型在更具挑战性的场景中始终优于 GPT-3.5 模型



(a) Easy tasks



(b) Medium tasks



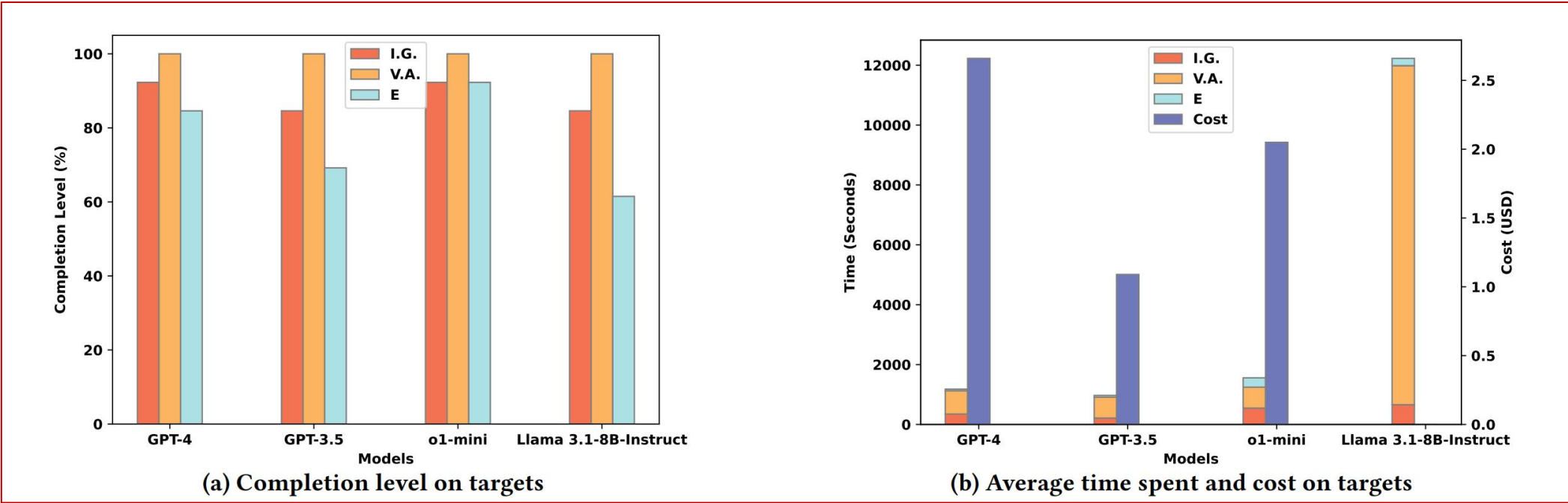
(c) Hard tasks

作者为了评估 PentestAgent 的效率，测量了执行渗透测试所需的时间和成本。

GPT-4：在各阶段的平均耗时分别为情报收集 346.7 秒、漏洞分析 780.9 秒、漏洞利用 52.3 秒，总累计时间为 1164.7 秒，每项任务的平均成本为 2.66 美元。

GPT-3.5：各阶段平均耗时为情报收集 212.9 秒、漏洞分析 698.8 秒、漏洞利用 58.6 秒，总平均时间为 1009.8 秒，每项任务成本为 1.09 美元。

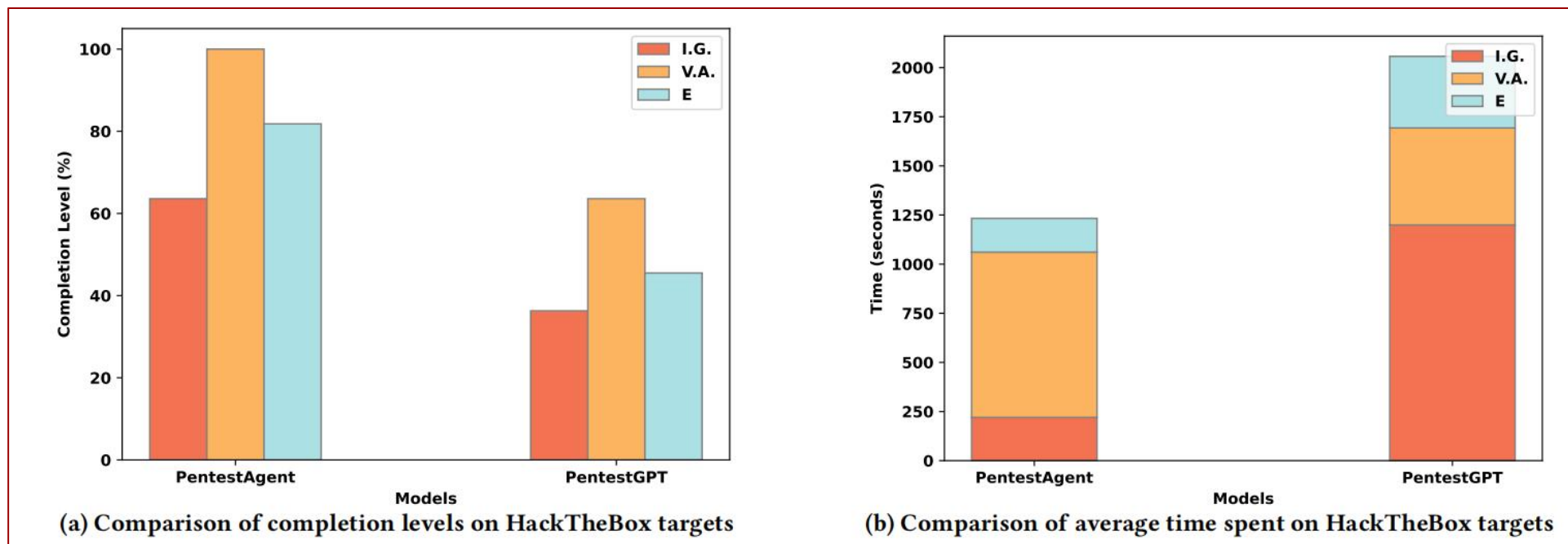
GPT-4 模型在自动化渗透测试中表现出更卓越的能力，但较GPT-3.5 模型运行成本更高



作者为了评估 PentestAgent 的效率，测量了执行渗透测试所需的时间和成本。随机选取 VulHub 10 个漏洞（5 easy/3 medium/2 hard）+ HackTheBox 11 个挑战（9 easy/1 medium/1 hard），并统一使用 GPT-3.5 以保证公平。

PentestAgent：完成度更高、整体更快，在 HackTheBox 上情报收集 220s vs 1199s、漏洞利用 172s vs 364s，并且各阶段完成水平普遍高于 PentestGPT。

PentestGPT：需要人工持续参与；虽漏洞分析可能略快，但其他阶段因人工开销更慢，导致总体效率与完成度不如 PentestAgent。



- PentestGPT → **任务树 + 分模块协作**，降低跑偏、保证连续性
 - PenHeal → **覆盖率/有效性/成本三目标**，用预算约束让修复更可落地
 - PentestAgent → **多智能体 + RAG**，端到端自动化、可量化（时间/成本）
-
- **优化上下文结构，减轻幻觉**。设计优于渗透测试任务树的结构，处理非线性攻击路径中的任务，设计更合理的记忆模块，存储长短期记忆。
 - **微观任务优化**。利用大型语言模型的推理、上下文理解和代码生成能力优化渗透测试任务中的细节，如指纹识别、密码推测等。
 - **合理规划多智能体架构，提高后渗透能力**。模拟人类团队，设计从初始访问到高效利用漏洞的全过程自动化架构，重点关注如何实现提权等后利用阶段。



感谢观看